Session:20

1 & 2. Product class with private attribute + getter/setter

```
class Product:
    def __init__(self, price):
        self._price = price  # private attribute (by convention)

    def display_price(self):
        print("Price:", self._price)

    # Getter
    def get_price(self):
        return self._price

    # Setter with validation
    def set_price(self, new_price):
        if new_price < 0:
            raise ValueError("Price cannot be negative")
        self._price = new_price
```

Example usage:

```
p = Product(100)
p.display_price()

print(p.get_price())

p.set_price(150)
print(p.get_price())

# p.set_price(-10)  # Raises ValueError
```

---

3. Playlist class with encapsulated song list

```
class Playlist:
    def __init__(self):
        self._songs = []  # private list of songs

    def add_song(self, song):
        self._songs.append(song)
        print(f"Added: {song}")

    def remove_song(self, song):
        if song in self._songs:
            self._songs.remove(song)
            print(f"Removed: {song}")
        else:
            print("Song not found")
```

```
    def get_songs(self):
        return list(self._songs)  # return a copy for safety
```

Example usage:

```
pl = Playlist()

pl.add_song("Song A")
pl.add_song("Song B")

print(pl.get_songs())

pl.remove_song("Song A")
print(pl.get_songs())
```

---

4. Abstract PaymentMethod with subclasses UPI and CreditCard

```
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Subclasses:

```
class UPI(PaymentMethod):
    def pay(self, amount):
        print(f"Processing UPI payment of ₹{amount}")


class CreditCard(PaymentMethod):
    def pay(self, amount):
        print(f"Processing Credit Card payment of ₹{amount}")
```

Example usage:

```
upi = UPI()
upi.pay(500)

card = CreditCard()
card.pay(1200)
```